

Project: Automated Lifecycle for 3-Tier Web Apps

1. Project Overview

You are tasked with architecting a fully automated CI/CD ecosystem for a standard 3-tier application (Frontend, Backend, and Database).

2. Technical Requirements

While you have the flexibility to choose your stack, the following "Golden Path" is recommended:

- **Version Control:** GitHub
 - **CI/CD Orchestration:** Jenkins (Pipeline-as-Code).
 - **Containerization:** Docker & GitHub Packages (Registry).
 - **Web Server/Proxy:** Nginx (configured for Blue/Green switching).
 - **Monitoring:** Prometheus & Grafana (Optional).
-

3. The Pipeline Architecture

The project is defined by three distinct automated workflows.

Pipeline 1: Continuous Integration

Trigger: Pull Request from `development` to `main/release`.

- **Code Linting:** Static analysis for both Frontend and Backend.
- **Unit Testing:** Run automated test suites.
- **Outcome:** Success allows the merge; failure adds comment to the PR.

Pipeline 2: Staging Deployment

Trigger: Merge/Commit to the `main` or `release` branch.

- **Build:** Create Docker images for Frontend and Backend.
- **Registry:** Tag images with a unique build number and push to GitHub Packages.

- **Provisioning:** Deploy the application to the **Staging Environment**.
- **Integration Testing:** Run API and UI tests against the live Staging environment.

Pipeline 3: Production Deployment (The Blue/Green Strategy)

Trigger: Manual Approval/Release Manager trigger.

- **Environment Preparation:** Identify the "Idle" environment (e.g., if Blue is live, prepare Green).
 - **Data Sync:** Replicate/Migrate database data to the new environment.
 - **Deployment:** Deploy the latest tagged images to the "Idle" environment.
 - **Smoke Tests:** Run health checks and integration tests on the new version *before* switching traffic.
 - **Traffic Swap:** Update the Nginx configuration to route traffic to the new environment.
 - **Rollback Logic:** Monitor health for 10 minutes; if failure occurs, automatically revert Nginx to the previous environment.
-

4. Monitoring & Self-Healing (optional)

To ensure high availability, you must implement a basic monitoring loop (via Jenkins or a dedicated tool):

1. **Health Checks:** Poll the Frontend and Backend endpoints every minute.
2. **Auto-Recovery:** If X consecutive health checks fail, the pipeline should trigger a re-provisioning of the affected container.

5. Required Deliverables

To successfully complete this capstone, students must provide the following three components:

GitHub Access	Repository links containing all application source code, Dockerfiles , and Pipeline-as-Code (e.g., Jenkinsfiles).
Live Demonstration	A functional walkthrough of the 3-pipeline lifecycle, demonstrating a code change moving from development to a successful Blue/Green production swap.
Project Documentation	A formal PDF document formatted for the accreditation commission, detailing the technical implementation.

6. Project Documentation Structure (PDF)

Front Matter

- **Cover Page:** Must include the Project Title, Course Name, Student Name, and Date.
- **Table of Contents:** A clear map of the document's chapters and sub-sections.

Chapter 1: Obiectivele proiectului

A high-level overview of what the project aims to achieve, focusing on automation, reliability, and the implementation of modern DevOps practices.

Chapter 2: Arhitectura soluției

A deep dive into the system design. This section must include a visual and narrative explanation of how the build pipelines integrate into the software development lifecycle (SDLC).

Chapter 3: Tehnologii utilizate

A detailed list of the stack chosen for the project and the rationale behind these choices.

Chapter 4: Pipeline Implementation (One section per pipeline)

For **Pipeline 1 (Validation)**, **Pipeline 2 (Staging)**, and **Pipeline 3 (Production)**, you must describe:

- **Trigger Mechanism:** When and how the pipeline is activated (e.g., Webhooks, PR merges, manual triggers).
- **Workflow Logic:** A step-by-step breakdown of the stages (Build, Test, Deploy).
- **Deployment Strategy:** For Pipeline 3, explain the logic used to manage the Blue/Green environment switch.

Annexes

- **Code Samples:** Key snippets of configuration files or critical logic.
- **Visual Evidence:** Screenshots of successful pipeline runs, monitoring dashboards, or the running application.